

Operating system principles 1

Paul Blondel

CNAM

September 1st, 2018

'(Kenneth) Thompson and (Dennis) Ritchie were among the first to realize that hardware and compiler technology had become good enough that an entire operating system could be written in C, and by 1978 the whole environment had been successfully ported to several machines of different types.'

Eric S. Raymond

- 1 Introduction
- 2 Main memory
- 3 Processor unit
- 4 Exchange units
- 5 Program production chain

- 1 Introduction
- 2 Main memory
- 3 Processor unit
- 4 Exchange units
- 5 Program production chain

How solving a problem?

solving a computer problem = execute instructions, one by one

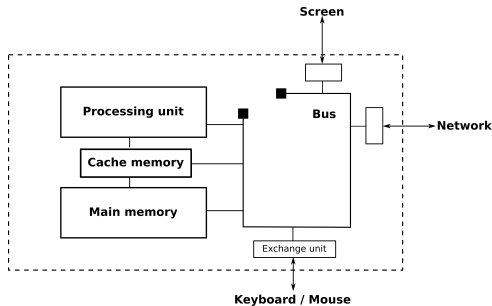
Cake recipe:

- Cook time 30 minutes
- 2 cups all-purpose flour
- 2 cups sugar
- 2 teaspoons baking powder
- 1 teaspoon salt
- 2 large eggs
- 200g chocolate
- Flour, sugar, salt and powder in one container
- Rest in another container
- Mix both and cook

Cake recipe (written as instructions):

- 1 **add** 2 cups flour *in* 1
- 2 **add** 2 cups sugar *in* 1
- 3 **add** 2 teaspoons powder *in* 1
- 4 **add** 1 teaspoon salt *in* 1
- 5 **add** 2 large eggs *in* 2
- 6 **add** 200g chocolate *in* 2
- 7 **mix** container 1
- 8 **mix** container 2
- 9 **mix** two containers
- 10 **cook** all 30 minutes

Most computers are organized following the **Von Neumann** architecture:



There are *four* main components:

- ① Main memory (RAM)
- ② Processor unit (CPU)
- ③ Peripheral control units
- ④ Communication bus

- 1 Introduction
- 2 Main memory
- 3 Processor unit
- 4 Exchange units
- 5 Program production chain

Why binary?

At low level, a computer machine works in **binary** basis:

- Numbers, characters, images, videos, ...: **all binary at low level**
- Underlying electronic components **can only manage 0s and 1s**

Decimal basis to binary basis:

$$1 = 2^0 = 1 \times 2^0 \text{ (simply: 1)}$$

$$10 = 8 + 2 = 2^3 + 2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \text{ (simply: 1 0 1 0)}$$

Binary basis to decimal basis:

$$1\ 1\ 1 = 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 4 + 2 + 1 = 7$$

Note

- Each 0 or 1 in a binary notation is called a **bit**
- Most significant bit: first bit (left), Least significant bit: last bit (right)

The more we have bits, the more combinations we can handle:

- With 2 bits:

- ▶ 0 0
- ▶ 0 1
- ▶ 1 0
- ▶ 1 1

- With 3 bits:

- ▶ 0 0 0
- ▶ 0 0 1
- ▶ 0 1 0
- ▶ 0 1 1
- ▶ 1 0 0
- ▶ 1 0 1
- ▶ 1 1 1

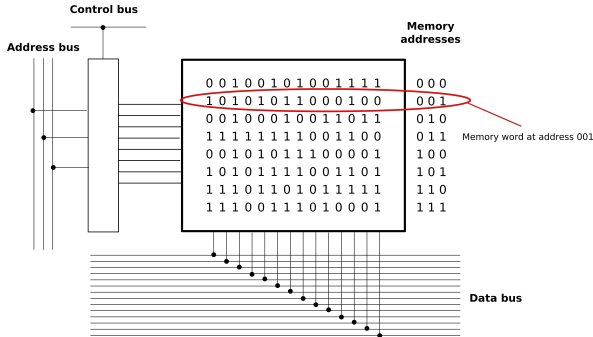
- With 4 bits:

- ▶ 0 0 0 0
- ▶ 0 0 0 1
- ▶ 0 0 1 0
- ▶ 0 0 1 1
- ▶ 0 1 0 0
- ▶ 0 1 0 1
- ▶ 0 1 1 1
- ▶ 1 0 0 0
- ▶ 1 0 0 1
- ▶ 1 0 1 1
- ▶ 1 1 0 0
- ▶ 1 1 0 1
- ▶ 1 1 1 1

Note

This is why **having more bits generally = more speed or memory**

The *main memory* works as follows (memory made of memory words):



- The *address bus* allows memory word selection (given an address)
- The memory word is returned by the *data bus* (here: word of 16 bits)
- The memory word is also written by the *data bus*
- The *control bus* handles read/write access (0 = read, 1 = write)

Instructions are...:

- ...stored **within memory words**
- ...can occupy **one or more memory words**
- ...**depend on the processor unit** (intel != ARM)
- ...made of **opcodes** and one or more **addressing mode(s)**:
 - ▶ *opcode*: binary code specifying an operation (add, mult, etc.)
 - ▶ *addressing mode*: binary address of other memory words with data

Examples:

add	address 1	address 2
0 0 1	1 0 1 1 0 1 1 1	0 0 1 1 0 1 1 0
jmp	address 3	
0 0 3	1 1 0 0 1 0 1 0	

Data are...

- ...also stored **within memory words**
- ...can also occupy **one or more memory words**

What is a bus?

Buses allow **information exchanges** and **data transfer**

There are three different buses:

- ① *address bus*: transport memory word addresses
- ② *control bus*: transport control operations (read/write for example)
- ③ *data bus*: transport data to read or write from memory

Note about the address bus

- The more bits, the more memory word addresses can be covered
- But memory size correlated to number of bits in memory words!

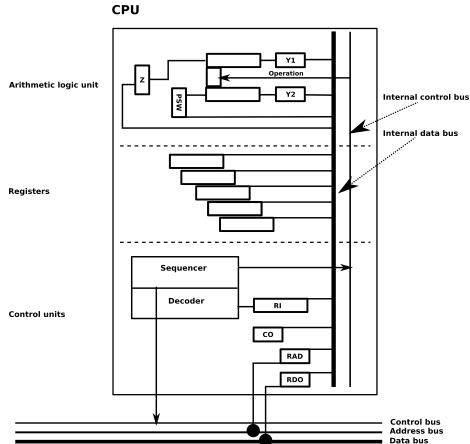
- 1 Introduction
- 2 Main memory
- 3 Processor unit**
- 4 Exchange units
- 5 Program production chain

What is the goal of the processor unit?

The processor unit **executes instructions** stored in the *main memory*

A processor unit (or CPU) is composed of three sub-components:

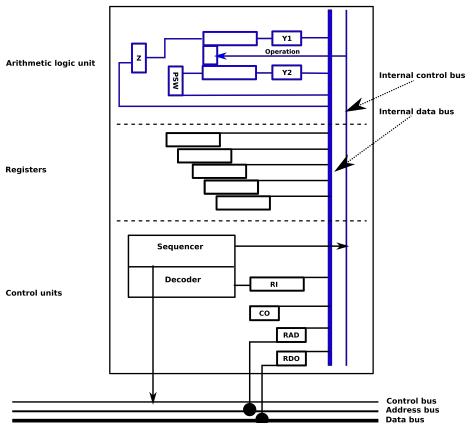
- 1 Arithmetic Logic Unit (ALU)
- 2 Registers
- 3 Control Units (CU)



Goal

Perform arithmetic operations on two values

CPU

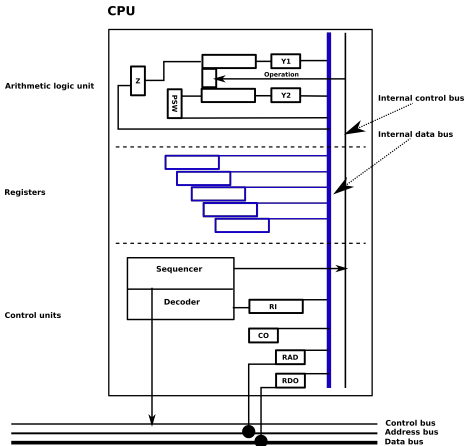


- 1st value placed in register $Y1^1$
- 2nd value placed in register $Y2^1$
- operation performed on the pair
- result placed in register Z^1
- PSW^1 : signal capacity overflows

¹ $Y1$, $Y2$, PSW and Z are *special* registers: they have a specific use

Goal

The registers unit store general information (with high speed access)

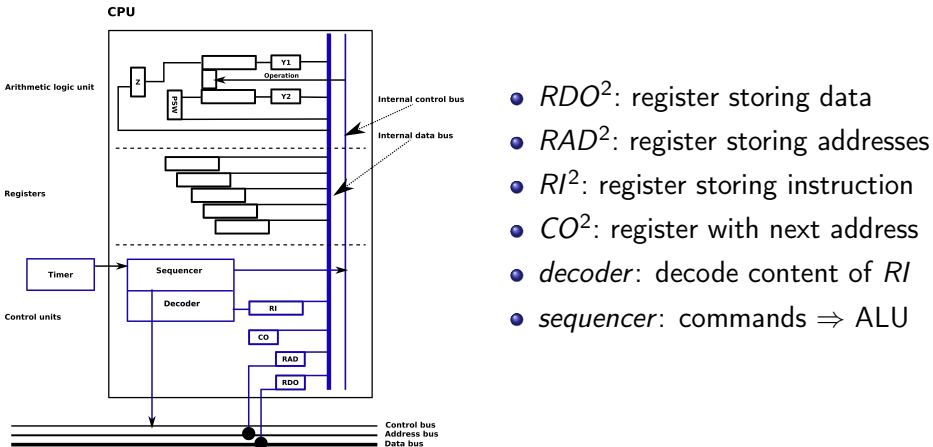


Registers are...

- ...filled by the internal data bus
- ...read from the internal data bus

Goal

Execute instructions using the *ALU* and the *registers unit*



²*RDO*, *RAD*, *RI* and *CO* are specific registers

With more details, when the CPU has to execute an instruction:

- 1 Content of **CO** is placed in **RAD**
- 2 **Read** is triggered thanks to the control bus
- 3 **RDO** receives instruction (data bus)
- 4 Content of **RDO** placed in **RI**
- 5 Instruction decoded in decoder
- 6 Set of commands sent (timer rythm) to **ALU** (sequencer)

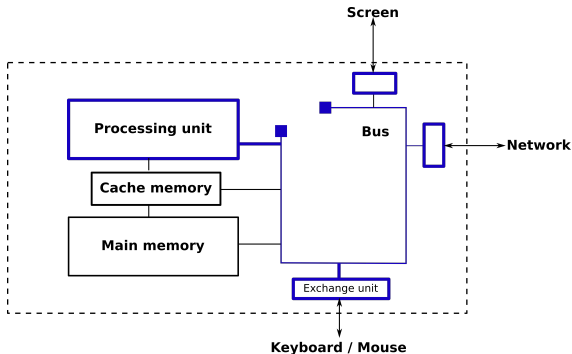
When the CPU has to read a data:

- 1 Address placed in **RDA**
- 2 **Read** is triggered thanks to control bus
- 3 **RDO** receives data
- 4 Data placed in general registers or **ALU** registers

- 1 Introduction
- 2 Main memory
- 3 Processor unit
- 4 Exchange units**
- 5 Program production chain

Exchange units...

- ...are in **communication with the processor unit** (internal bus)
- ...are also in **communication with the peripherals**



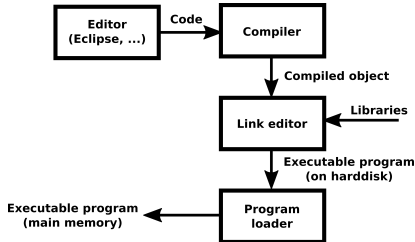
Goal

- Ensure standard comm. between *processor unit* and *peripherals*
- Exchange unit: takes care of variety of hardware

- 1 Introduction
- 2 Main memory
- 3 Processor unit
- 4 Exchange units
- 5 Program production chain**

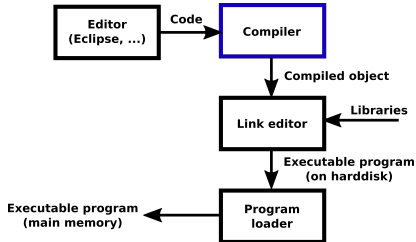
The *program production chain* is at follows:

- 1 High-level code is written using an **editor**
- 2 Code is **compiled** into binary objects
- 3 Objects are **linked** together and with libraries
- 4 Program is **loaded in main memory**



What is compilation?

It consists in transforming **human-readable source code** into **binary code**



Compiling consists in four steps:

- 1 the **lexical analysis**
- 2 the **syntactic analysis**
- 3 the **semantic analysis**
- 4 the **code optimization** and **code generation**

What is the lexical analysis?

lexical analysis = **recognizing the words of the language**

A high-level programming language is made of:

- an **alphabet** (set of authorized symbols)
 - ▶ characters, numbers, punctuation, etc.
 - ▶ examples: !?=/{};:-()abcdef..z1234..9
- **words** (made of elements of the alphabet):
 - ▶ examples: int,char,const,etc.
- **phrases**:
 - ▶ **groups** of language words
 - ▶ respecting a set of **grammatical rules**

Examples of *grammatical rules* (following the rules of Backus-Naur):

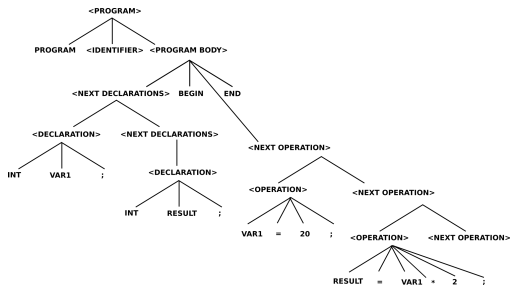
<PROGRAM> => PROGRAM <IDENTIFIER> <PROGRAM BODY>
<PROGRAM BODY> => <OTHER DECLARATIONS> BEGIN <NEXT DECLARATIONS> END
<NEXT DECLARATIONS> => <DECLARATION> or
<DECLARATION> <NEXT DECLARATIONS>
<DECLARATION> => INT <IDENTIFIER>;
<TERM> => <INTEGER> or <IDENTIFIER>
<OPERATOR> => + or - or * or /
<IDENTIFIER> => <CHARACTER> or <CHARACTER> <NUMBER>
<INTEGER> => <NUMBER> or <NUMBER> <CHARACTER>
<OPERATION> => <IDENTIFIER> = <TERM> <OPERATOR>
<TERM>; or <IDENTIFIER> = <TERM>;
<NEXT OPERATION> => <OPERATION> or <OPERATION>
<NEXT OPERATION>
<CHARACTER> => A or B or C or D, ..., Z
<NUMBER> => 0 or 1 or 2 or 3, ...9

What does the syntactic analysis?

Analyze symbols returned by the lexical analysis and **check the syntax**

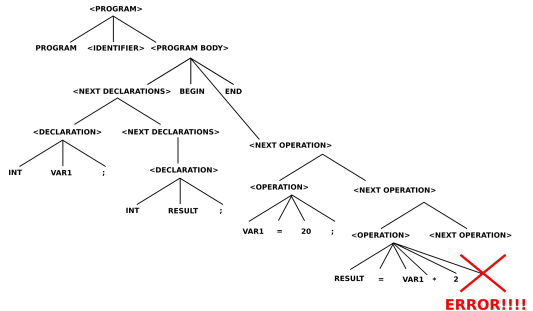
Example 1 (program with correct syntax):

```
PROGRAM myProgram
INT VAR1;
INT RESULT;
BEGIN
VAR1 = 20;
RESULT = VAR1 * 2;
END
```



Example 2 (program with a syntax error):

```
PROGRAM myProgram
INT VAR1;
INT RESULT;
BEGIN
VAR1 = 20;
RESULT = VAR1 * 2
END
```



What does the semantic analysis?

It associates a purpose to the phrases by:

- recognizing the objects and their **properties** (type, time of life, etc.)
- checking that **objects are properly used**

Examples of common object misusages detected by semantic analysis:

- objects declared several times
- objects used whereas deleted before
- object declared with wrong type
- etc...

What is the *code generation and optimization*?

The final step of the *compilation* generating a *binary relocatable file*³

It starts by getting a **table of symbols** of the program, example:

Object name	Type	Size	Address
VAR1	INT	4 bytes	(0)
RESULT	INT	4 bytes	(4)

Then, by transforming the code into an **intermediary code**:

Original code:

```
PROGRAM myProgram
INT VAR1;
INT RESULT;
BEGIN
VAR1 = 20;
RESULT = VAR1 * 2
END
```

Intermediary code:

```
(0)
(4)
INIT (0) 20
MULT (4) (0) 2
STOP
```

³relocatable means: the addresses of the objects start at 0

The *intermediary code* is then optimized:

Intermediary code:

```
(0)
(4)
INIT (0) 20
MULT (4) (0) 2
STOP
```

Optimized intermediary code:

```
(0)
(4)
INIT (0) 20
INIT (4) 40
STOP
```

What is code optimization?

It is reducing constant expressions, simplifying for and while loops, etc.

The final step is transforming the *intermediary code* into *binary code*:

Optimized intermediary code:

```
(0)
(4)
INIT (0) 20
INIT (4) 40
STOP
```

Binary code:

```
0000 000 0001 00000000000010100
0001 001 0001 00000000000000000
0000 000 0001 00000000000101000
0001 001 0001 00000000000000100
```

After *compilation*, programs may be split in several binary files⁴, example:

Let's consider an application to search for hotels.

The source code has been split into three files:

- ❶ *main.c*: containing main program code
- ❷ *display.c*: containing code to display windows
- ❸ *search.c*: containing code to search hotels

After the compilation you will have three separate binary objects:

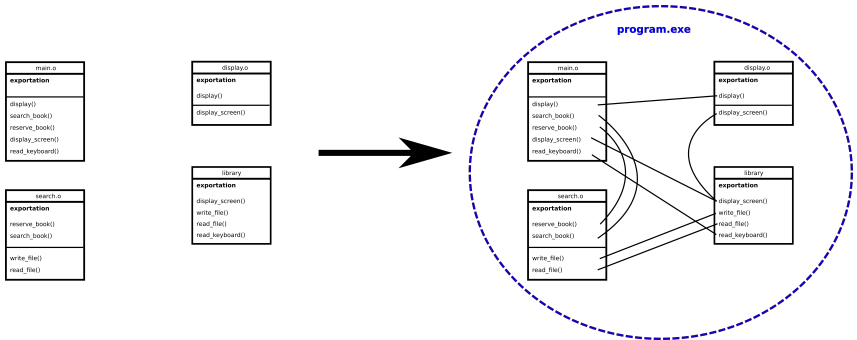
- ❶ *main.o*: main program binary object
- ❷ *display.o*: display object
- ❸ *search.o*: search object

⁴This happens when the source code is dispatched in several files

What does link editing?

It links the components of the objects and make one executable⁵

Example:



⁵The linker also link objects with external libraries (for instance, libraries to display windows)

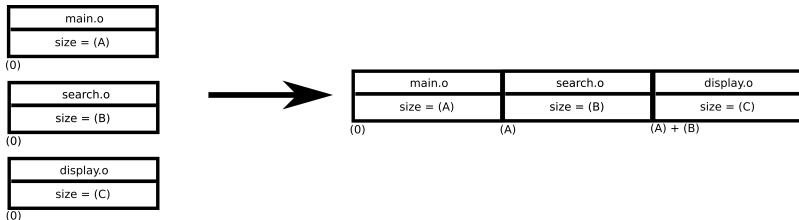
Concretely the link editing process is as follows:

- ① building a map of implementation
- ② building a table of links
- ③ handling external libraries
- ④ building the final executable

What is a map of implementation?

It gives the **new starting addresses of each object**⁵

Example of a building of a map of implementation:



⁵Indeed, originally each module has (0) as a starting address

What is a table of links?

It **links the components** of the objects using the *map of implementation*

Example:

Before links are made:

Name of component	Addresses
search_book	UNDEFINED
display	UNDEFINED
reserve_book	UNDEFINED
read_keyboard	UNDEFINED
display_screen	UNDEFINED
read_file	UNDEFINED
write_file	UNDEFINED

After links are made:

Name of component	Addresses
search_book	(A) + address_search_book
display	(A) + (B) + address_display
reserve_book	(A) + address_reserve_book
read_keyboard	UNDEFINED
display_screen	UNDEFINED
read_file	UNDEFINED
write_file	UNDEFINED

What is a table of links?

It **links the components** of the objects using the *map of implementation*

Example:

Before links are made:

Name of component	Addresses
search_book	UNDEFINED
display	UNDEFINED
reserve_book	UNDEFINED
read_keyboard	UNDEFINED
display_screen	UNDEFINED
read_file	UNDEFINED
write_file	UNDEFINED

After links are made:

Name of component	Addresses
search_book	(A) + address_search_book
display	(A) + (B) + address_display
reserve_book	(A) + address_reserve_book
read_keyboard	UNDEFINED
display_screen	UNDEFINED
read_file	UNDEFINED
write_file	UNDEFINED

Some links are not yet resolved!

External libraries are not yet linked!

External libraries can be linked in two ways:

- **statically**
- or later **when loaded in *main memory***

When *external libraries* are statically linked:

- **external libraries are added** to the final executable⁶
- the table of links is then updated as below⁷

Name of component	Addresses
search_book	(A) + address_search_book
display	(A) + (B) + address_display
reserve_book	(A) + address_reserve_book
read_keyboard	(A) + (B) + (C)
display_screen	(A) + (B) + (C) + (D)
read_file	(A) + (B) + (C) + (D) + (E)
write_file	(A) + (B) + (C) + (D) + (E) + (F)

⁶The final executable becomes much bigger in size when statically linked

⁷(D) = size *read_keyboard* library, (E) = size *display_screen* library, (F) = size *read_file* library

What does the loader?

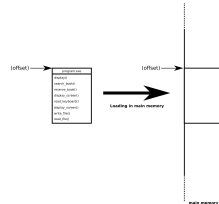
The loader...

- ...changes the first address of the executable
- ...recalculate internal component addresses
- ...load the executable in *main memory*

Loading the new executable:

New addresses:

Name of component	Addresses
	(offset)
search_book	(offset) + (A) + address_search_book
display	(offset) + (A) + (B) + address_display
reserve_book	(offset) + (A) + address_reserve_book
read_keyboard	(offset) + (A) + (B) + (C)
display_screen	(offset) + (A) + (B) + (C) + (D)
read_file	(offset) + (A) + (B) + (C) + (D) + (E)
write_file	(offset) + (A) + (B) + (C) + (D) + (E) + (F)



Now, the executable can be executed by the *processor unit*⁷!

⁷The first address of the executable in *main memory* is read by the processor unit

If libraries are dynamically linked, the loader will first:

- 1 load independently the libraries in *main memory*
- 2 change the addresses in the table of links accordingly

Example:

Name of component	Addresses
	(offset)
search_book	(offset) + (A) + address_search_book
display	(offset) + (A) + (B) + address_display
reserve_book	(offset) + (A) + address_reserve_book
read_keyboard	(offset read_keyboard)
display_screen	(offset display_screen)
read_file	(offset read_file)
write_file	(offset write_file)